

Laboratorio 4 - Parte 2: REST API Blueprints mediante Autenticación JWT y Spring Boot

María Paula Rodríguez Muñoz
Juan Andrés Suárez
Juan Pablo Nieto
Tomás Felipe Ramírez

1. Introducción

Este laboratorio extiende un proyecto previo de una REST API para la gestión de blueprints. Su objetivo es implementar autenticación y autorización mediante JWT con criptografía RSA usando Spring Boot 3.3.x y Java 21. La aplicación sigue una arquitectura por capas e incluye procesamiento de datos para optimizar la información geométrica. La persistencia soporta almacenamiento en memoria y base de datos PostgreSQL.

2. Objetivos

2.1. Objetivo General

Desarrollar una API REST para la gestión de blueprints que implemente autenticación segura con JWT, control de acceso basado en scopes y una capa de persistencia flexible que permita utilizar diferentes backends de almacenamiento.

2.2. Objetivos Específicos

- Implementar una arquitectura por capas que facilite la organización y mantenibilidad del sistema.
- Integrar Spring Security para la validación de tokens JWT firmados con RSA.
- Configurar autenticación de usuarios con contraseñas cifradas mediante BCrypt.
- Implementar control de acceso mediante scopes para las operaciones sobre blueprints.
- Desarrollar persistencia con soporte para almacenamiento en memoria y PostgreSQL.
- Incorporar filtros para el procesamiento de blueprints.
- Documentar los endpoints con OpenAPI y realizar pruebas unitarias.

2.3. Diagrama de Componentes

3. Ejecución del Proyecto

3.1. Requisitos Previos

- Java 21 (o superior) instalado.
- Maven 3.9+ instalado.
- PostgreSQL 15+ accesible (localmente o via Docker).
- Docker (opcional, para containerización).

3.2. Configuración de Base de Datos

La base de datos PostgreSQL se configura mediante Docker:

```
docker run -d --name postgres-blueprints \  
-e POSTGRES_DB=blueprints \  
-e POSTGRES_USER=postgres \  
-e POSTGRES_PASSWORD=postgres \  
-p 5432:5432 \  
postgres:15
```

La aplicación usa Spring SQL init mode para ejecutar automáticamente `schema.sql` al iniciar, creando tablas si no existen.

3.3. Compilación y Ejecución

Desde el directorio raíz del proyecto:

```
# Compile  
mvn clean install  
  
# Ejecutar  
mvn spring-boot:run
```

La aplicación iniciará en `http://localhost:8080`.

3.4. Endpoints Principales

Todas las operaciones sobre `/api/**` requieren autenticación JWT (excepto login).

3.4.1. Autenticación

```
POST /auth/login
Content-Type: application/json

{
  "username": "student",
  "password": "student123"
}

# Respuesta
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 7200
}
```

3.4.2. Obtener Todos los Blueprints

```
GET /api/v1/blueprints
Authorization: Bearer <access_token>
```

3.4.3. Obtener Blueprints de un Autor

```
GET /api/v1/blueprints/john
Authorization: Bearer <access_token>
```

3.4.4. Obtener un Blueprint Específico

```
GET /api/v1/blueprints/john/house
Authorization: Bearer <access_token>
```

3.4.5. Crear un Blueprint

```
POST /api/v1/blueprints
Authorization: Bearer <access_token>
Content-Type: application/json

{
  "author": "john",
  "name": "kitchen",
  "points": [
    {"x": 1, "y": 1},
    {"x": 10, "y": 1},
    {"x": 10, "y": 10}
  ]
}

# Respuesta: 201 Created
```

3.4.6. Agregar un Punto a un Blueprint

```
PUT /api/v1/blueprints/john/kitchen/points
Authorization: Bearer <access_token>
Content-Type: application/json

{"x": 5, "y": 5}

# Respuesta: 202 Accepted
```

3.5. Ejecución de Pruebas

```
# Ejecutar todas las pruebas
mvn clean test

# Generar reporte de cobertura
# El reporte se ubica en: target/site/jacoco/index.html
open target/site/jacoco/index.html
```

SpringBoot_REST_API_Blueprints

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
edu.eci.arsw.blueprints.controllers		57 %		0 %	8	17	21	53	4	13	2	6
edu.eci.arsw.blueprints.security		40 %		0 %	8	17	15	36	6	15	0	4
edu.eci.arsw.blueprints.persistence.impl		85 %		100 %	4	15	5	51	4	10	0	1
edu.eci.arsw.blueprints.config		7 %		n/a	1	2	11	12	1	2	0	1
edu.eci.arsw.blueprints		37 %		n/a	1	2	2	3	1	2	0	1
edu.eci.arsw.blueprints.filters		96 %		81 %	3	14	2	23	0	6	0	3
edu.eci.arsw.blueprints.model		97 %		70 %	3	13	0	15	0	8	0	2
edu.eci.arsw.blueprints.persistence		100 %		100 %	0	14	0	32	0	11	0	3
edu.eci.arsw.blueprints.services		100 %		n/a	0	6	0	11	0	6	0	1
Total	334 of 1.331	74 %	18 of 54	66 %	28	100	56	236	16	73	2	22

Figura 1: Jacoco

4. Evidencias de Ejecución

En esta sección se presentan las evidencias del funcionamiento de la API REST protegida mediante autenticación JWT y control de acceso basado en scopes.

4.1. Usuarios configurados y permisos

El sistema de seguridad cuenta con dos usuarios configurados con diferentes permisos (scopes) para controlar el acceso a los endpoints de la API.

- **student**: Permisos de lectura y escritura sobre blueprints.
- **assistant**: Permisos únicamente de lectura.

Los usuarios configurados en el sistema de seguridad son:

Usuario	Contraseña	Permisos (Scopes)
student	student123	Lectura y Escritura (blueprints.read, blueprints.write)
assistant	assistant123	Solo Lectura (blueprints.read)

Figura 2: Usuarios configurados y scopes asignados en el sistema de seguridad

4.2. Documentación de la API con OpenAPI (Swagger)

La API se documenta automáticamente mediante OpenAPI, permitiendo visualizar y probar los endpoints disponibles desde una interfaz gráfica.

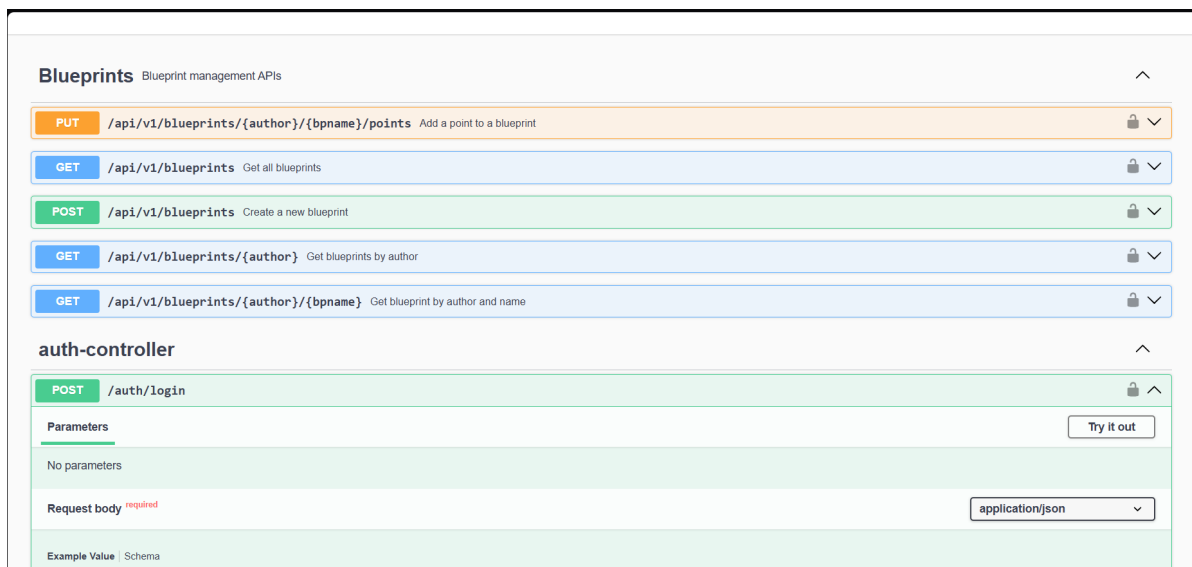


Figura 3: Documentación de endpoints disponibles en Swagger

4.3. Proceso de autenticación JWT

Para acceder a los endpoints protegidos es necesario autenticarse mediante el endpoint `/auth/login`. Este endpoint recibe un usuario y contraseña y retorna un token JWT firmado mediante RSA.

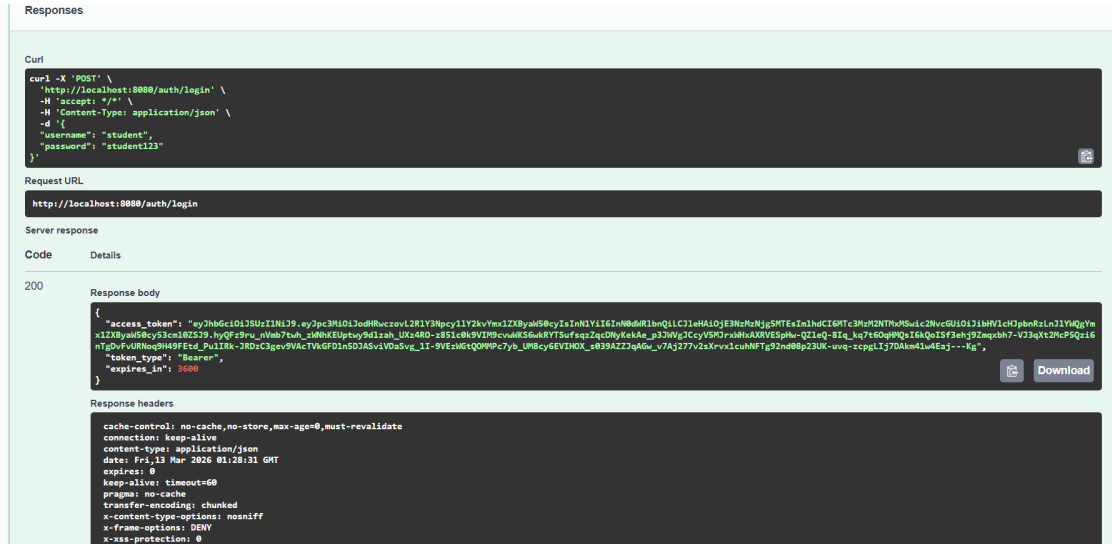


Figura 4: Solicitud de autenticación al endpoint `/auth/login`

Una vez autenticado correctamente, el sistema genera un token JWT que será utilizado para acceder a los endpoints protegidos.

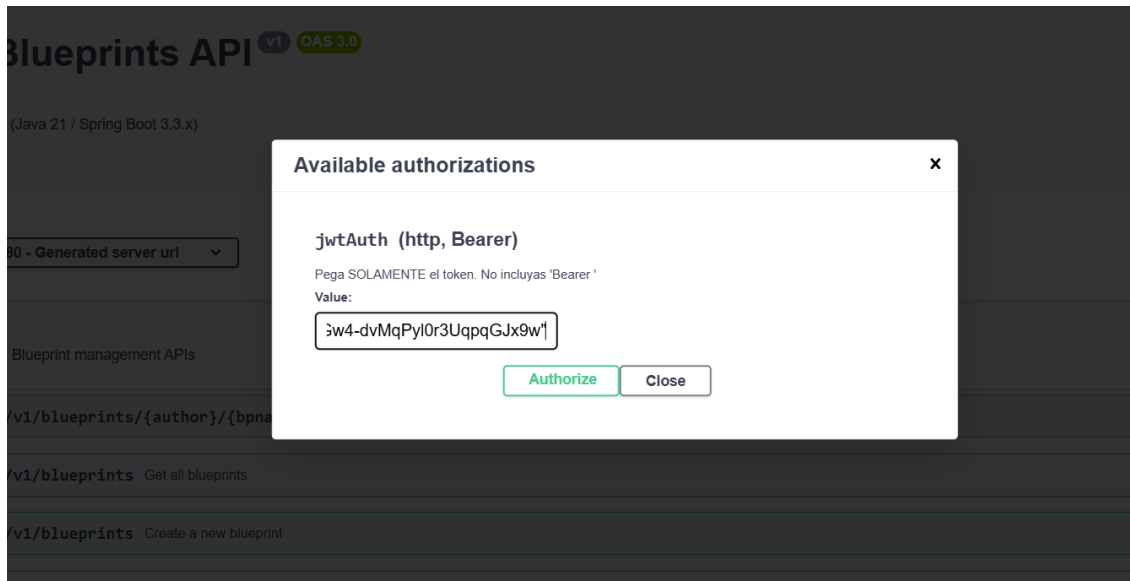
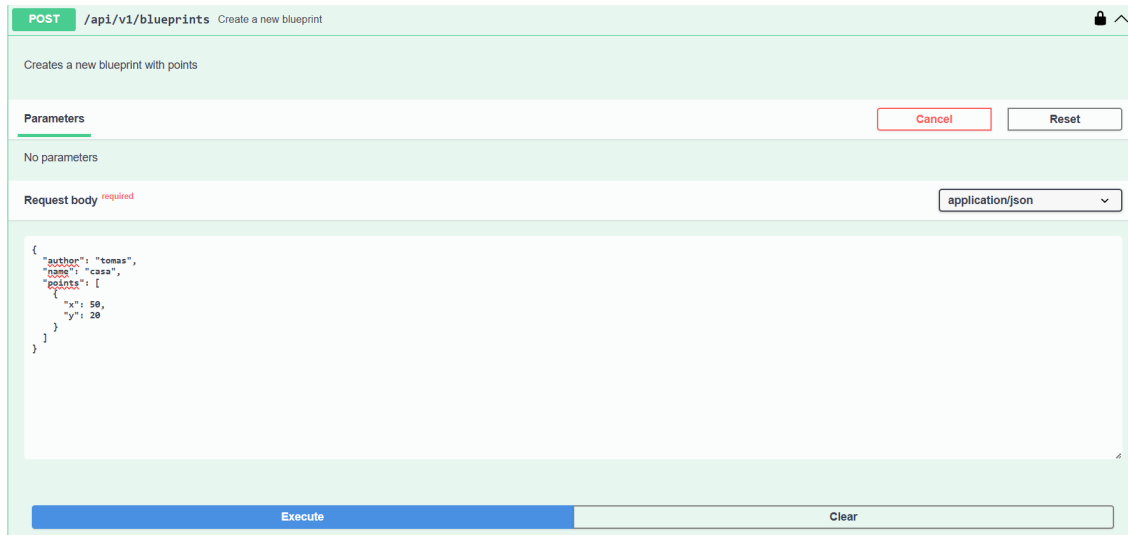


Figura 5: Token JWT generado después de autenticación exitosa

4.4. Creación de un Blueprint

Con el token JWT autorizado, se puede acceder al endpoint protegido `/api/v1/blueprints` para crear un nuevo blueprint.

En este caso se crea un blueprint con autor *tomas*, nombre *casa* y un punto geométrico inicial.



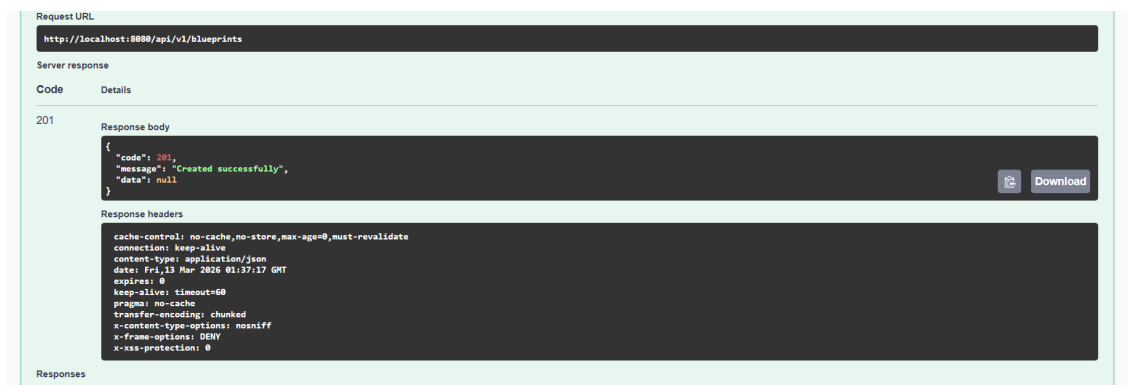
The screenshot shows a REST client interface for a POST request to `/api/v1/blueprints`. The header bar indicates the method is POST and the endpoint is `/api/v1/blueprints` with the description "Create a new blueprint". Below the header, there's a description: "Creates a new blueprint with points". The "Parameters" section is empty, showing "No parameters". The "Request body" section is marked as "required" and has a dropdown menu set to "application/json". The request body contains a JSON object:

```
{  "author": "tomas",  "name": "casa",  "points": [    {      "x": 50,      "y": 20    }  ]}
```

 At the bottom, there are "Execute" and "Clear" buttons.

Figura 6: Solicitud POST para crear un nuevo blueprint

El servidor responde con código HTTP **201 Created**, indicando que el blueprint fue creado correctamente.



The screenshot shows the server response for the POST request. The "Request URL" is `http://localhost:8080/api/v1/blueprints`. The "Server response" section shows a "Code" of 201 and "Details". The "Response body" is a JSON object:

```
{  "code": 201,  "message": "Created successfully",  "data": null}
```

 There is a "Download" button next to the response body. The "Response headers" section lists various headers:

```
cache-control: no-cache, no-store, max-age=0, must-revalidate
connection: keep-alive
content-type: application/json
date: Fri, 13 Mar 2020 01:37:17 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-ssr-protection: 0
```

 At the bottom, there is a "Responses" section.

Figura 7: Respuesta exitosa del servidor al crear un blueprint

4.5. Consulta de Blueprints por Autor

También se verificó el funcionamiento del endpoint que permite consultar los blueprints registrados para un autor específico.

GET /api/v1/blueprints/{author} Get blueprints by author

Returns all blueprints for a specific author

Parameters

Name	Description
author * required string (path)	tomas

Execute Clear

Figura 8: Consulta de blueprints filtrados por autor

El sistema retorna correctamente la información almacenada en el sistema.

Request URL
http://localhost:8080/api/v1/blueprints/tomas

Server response

Code Details

200

Response body

```
{
  "code": 200,
  "message": "Success",
  "data": [
    {
      "author": "tomas",
      "name": "casa",
      "points": [
        {
          "x": 50,
          "y": 20
        }
      ]
    }
  ]
}
```

Response headers

```
cache-control: no-cache, no-store, max-age=0, must-revalidate
```

Figura 9: Respuesta del servidor con los blueprints registrados

Estas pruebas demuestran que el sistema de autenticación JWT funciona correctamente, permitiendo acceder a los endpoints protegidos y ejecutar operaciones sobre los blueprints almacenados.

5. Conclusiones

- Se logró implementar una API REST segura y funcional utilizando tecnologías del ecosistema Java y Spring Boot.
- La integración de Spring Security con JWT y firma RSA permitió establecer un mecanismo de autenticación robusto y escalable.
- La arquitectura por capas y el uso de inyección de dependencias facilitaron la organización, mantenibilidad y escalabilidad del código.
- El diseño de una capa de persistencia abstracta permitió soportar diferentes backends de almacenamiento como memoria y PostgreSQL.

- El uso de herramientas como Spring Boot, Maven, Docker y OpenAPI permitió desarrollar y documentar la aplicación siguiendo buenas prácticas de desarrollo.
- La implementación realizada sienta bases para futuras mejoras como autenticación avanzada, integración con proveedores externos y optimización del sistema.